Audit: Separation of Repo Intelligence from Script Generation

Date: 2026-06-26
Scope: Establish a strict separation of responsibilities — Repo Intelligence owns **all** repository conventions; Script Generation only generates code using that information. No new intelligence beyond convention ownership. Zero regressions.

TL;DR

The good news: the platform **already** has a canonical, DB-cached `RepositoryProfile` (Repo Intelligence) and a distillation layer (`analyzeRepoStructure → RepoStructureAnalysis` , plus a `FolderStructureAnalyzer` placement service). Script Generation **already** honours the repo's detected **test folder**, **page-object folder**, and **file-naming conventions**.

The bad news: a handful of **architectural decisions are still hardcoded inside Script Generation** — most visibly the **test-data folder** (`tests/data`), the **fixtures folder** (`fixtures/`), and the **helpers/utils folder** (`utils/`). There is also **no convention** for the **API folder**, **import style**, or **test-data format**, and the convention logic is **physically split** across `src/script-gen/` and `src/services/` rather than owned by Repo Intelligence.

This audit fixes that by introducing a single **Project Convention Profile** owned by Repo Intelligence, deriving it purely from the cached `RepositoryProfile` , and routing every folder/placement decision through it — with defaults equal to today's hardcoded values so existing behaviour is unchanged (zero regressions) and only **connected repos with different conventions** see corrected placement.

Q1. What architectural decisions is Script Generation making that should belong to Repo Intelligence?

All of the following live in `src/script-gen/script-gen-engine.ts` :

Decision	Where (today)	Should be
Test-data folder = `tests/data`	`buildTestDataModule()` returns `path: 'tests/data/test-data.ts'`; `generateFromTestCase()` imports `./data/test-data`; comparison at `generateFromTestCases()`	Repo Intelligence (`testDataFolder`)
Fixtures folder = `fixtures/`	`generatePomFiles()` emits `fixtures/test-fixtures.ts` and `fixtures/auth.ts` (uses `hasFixtures` boolean, ignores the detected path)	Repo Intelligence (`fixtureFolder`)
Helpers/utils folder = `utils/`	scaffold spec `utils/test-helpers.ts` (uses `hasUtils` boolean, ignores the detected path)	Repo Intelligence (`helperFolder`)
Greenfield folder fallbacks = `pages` , `tests`	`generatePomFiles()` : `pageDir = analysis ? … : 'pages'` , `testDir = analysis ? … : 'tests'`	Centralised convention defaults
Test-data import style = relative `./data/test-data`	hardcoded import string in spec body	Derived from resolved folders + `importStyle`
Test-data format = always a `.ts` module	`buildTestDataModule()` always emits TypeScript	Repo Intelligence (`testDataPattern`) — exposed, emission unchanged
Page-object pattern = always a `class`	`generatePageObject()`	Repo Intelligence (`pageObjectPattern`) — exposed, already matches

> Note: spec-file placement (`testDir`) and page-object placement (`pageObjectDir`) + file naming are **already** profile-driven via `analyzeRepoStructure()` . The defects are concentrated in **test-data, fixtures, and utils** placement, which bypass the profile.

Q2. What repository conventions are already detected today?

Built by `src/context/repository-context-engine.ts` into `RepositoryProfile` (`src/context/types.ts`) and consumed via `analyzeRepoStructure()` :

- **Framework / language / test pattern** (`flat-scripts | page-object-model | hybrid`) / locator strategy.
- **Folder structure**: `testFolder` , `pageObjectFolder` , `fixtureFolder` , `utilsFolder` , `configFiles` , `supportFiles` .
- **File-naming convention**: casing (snake/kebab/camel/Pascal), separator, extension/suffix, numeric-prefix detection, next file number — for both specs and page objects.
- **Coding style**: quotes, semicolons, indent, tag convention.
- **Reusable assets**: `pageObjects` , `helperFunctions` , `fixtures` , `customCommands` , `dataFiles` (with `type`), `sharedConstants` .
- **Scaffold presence**: Playwright/Cypress config, CI workflow, README, `.env` /dotenv.
- **Credential style**: inline / env / fixture.

These are **persisted** (DB-cached) and already drive spec + page-object output.

Q3. What repository conventions are still hardcoded / missing?

1. **`testDataFolder`** — no field in `FolderStructure` ; generator hardcodes `tests/data` .
2. **`apiFolder`** — not detected anywhere.
3. **`importStyle`** (`relative` vs `alias` / `absolute` like `@/pages/...`) — not detected; import-path builder is always relative.
4. **`testDataPattern`** (`json` | `ts` | `factory` | …) — not surfaced as a convention; `dataFiles[].type` exists but is unused for generation.
5. **Fixture & helper folder paths** — detected as **booleans** (`hasFixtures` , `hasUtils`) but the path is ignored, so output is hardcoded `fixtures/` and `utils/` .
6. **Defaults are duplicated** across call sites instead of living in one place.

Q4. Where should the Project Convention Profile be built and cached?

It already has the right home — we formalise it, we do not add a second cache.

- **Built by** `repository-context-engine.ts::buildRepositoryProfile()` during a repo scan.
- **Cached in** PostgreSQL `repository_context` (JSONB columns: `folder_structure` , `coding_style` , `page_objects` , `fixtures` , …) via `repo-scan-service.ts` , read back through `getRepositoryContext(repoId, companyId, projectId)` in `src/db/postgres.ts` , keyed by **repo + branch + company** with `profile_version` / `last_scanned_at` .
- The **Project Convention Profile** is a **pure, derived view** of this cached `RepositoryProfile` — it is **not** separately persisted. The

`RepositoryProfile` remains the single source of truth (consistent with the product philosophy: derive, don't duplicate). Deriving it is cheap and deterministic, so it can be computed on demand wherever a feature has the profile in hand.

Q5. Which APIs should Script Generation call instead of deciding for itself?

A single canonical resolver owned by Repo Intelligence:

```
// src/intelligence/project-convention-profile.ts
buildConventionProfile(profile: RepositoryProfile | null): ProjectConventionProfile
```

`ProjectConventionProfile` answers every "where / which" question:

Script Gen asks…	Profile field / helper
Where does this test go?	`testFolder`
Where does the Page Object go?	`pageObjectFolder`
Where does test data go?	`testDataFolder` + `testDataPath(fileName)`
Where do fixtures go?	`fixtureFolder` + `fixturePath(fileName)`
Which helper folder?	`helperFolder` + `helperPath(fileName)`
Where does API code go?	`apiFolder`
Which naming convention?	`namingConvention` (+ existing `RepoStructur-eAnalysis.naming`)
Which import style?	`importStyle` + `importSpecifier(fromDir, toModule)`
Which test-data format?	`testDataPattern`
Which PO pattern?	`pageObjectPattern`

Existing `analyzeRepoStructure()` and `FolderStructureAnalyzer` continue to power spec/page-object naming and placement; the convention profile **wraps and consolidates** them so Script Generation has **one** thing to call.

Q6. Refactor — Repo Intelligence owns conventions; Script Generation consumes

Principle: defaults equal today's hardcoded values, so greenfield generation and the self-contained Test-Case-Lab bundle are **byte-for-byte unchanged**. The profile only changes output when a **connected repo** genuinely uses a different convention (e.g. root-level `data/` instead of `tests/data`) — which is exactly the bug shown in the SauceDemo screenshot.

Changes (all additive / zero-regression):

1. **`src/context/types.ts`** — add `testDataFolder` and `apiFolder` to `FolderStructure`.
2. **`src/context/repository-context-engine.ts`** — detect `testDataFolder` (`data`, `test-data`, `testdata`, `tests/data`, `fixtures/data`) and `apiFolder` (`api`, `apis`, `services`, `endpoints`). Existing fixture/utils detection untouched.
3. **`src/intelligence/project-convention-profile.ts`** (new) — the canonical `ProjectConventionProfile` + `buildConventionProfile()` resolver with the exact shape requested (framework, folders, patterns, importStyle, namingConvention, testDataPattern) and path/import helpers. Pure function, safe defaults when no profile.
4. **`src/script-gen/repo-analyzer.ts`** — extend `RepoStructureAnalysis` with `testDataDir`, `fixtureDir`, `helperDir`, `apiDir`, `importStyle`, `testDataPattern`, derived from the profile (defaults preserve current values).
5. **`src/script-gen/script-gen-engine.ts`** — replace the hardcoded `tests/data`, `fixtures/`, `utils/` literals with values resolved from the convention profile; compute the spec→test-data import **relative to the resolved folders** (default resolves to today's `./data/test-data`).

Out of scope (deliberately not rewired, to avoid adding new behaviour): emitting test data as JSON instead of a `.ts` module, and switching generated imports to alias/`@` style — these are exposed as conventions on the profile (so Healing, PR Generation, Migration, etc. can consume them later) but the generator's emission format is unchanged. `importStyle` defaults to `relative`, so the import builder behaves exactly as before unless a repo clearly uses aliases.

Future reuse: the same `ProjectConventionProfile` is now the one object that Healing, Repo Patching, the Learning Engine, PR Generation, the Migration Assistant, Framework Conversion, and Component Intelligence consult — no feature guesses repository structure independently.